Chapter 10

Loading Layers and Rendering Feature Layers

This chapter will show how to load feature classes and raster datasets into ArcMap as layers, as well as how to create simple fill renderer to render polygon feature layers. A number of related ArcObjects components including layer, feature class, raster dataset, workspace, renderer, symbol, and color will be closely examined against ArcObjects model diagrams. You will follow examples to create and implement an add-in button in the CityGIS project to load vector and raster layers. You will also continue to practice object model diagram analysis skills and learn more VB.NET and ArcObjects programming techniques.

1. Loading Feature Layers

1.1 Object model diagram analysis and solution development

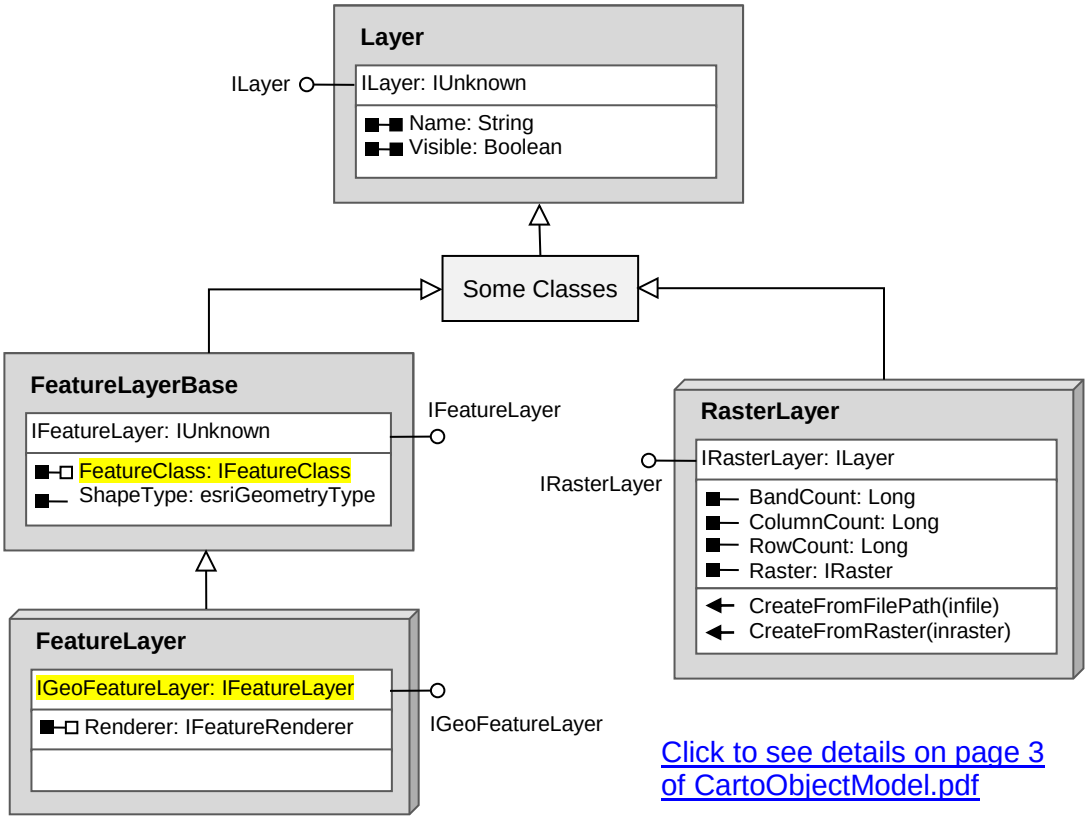


Figure 1 The abstract Layer class and some of its child co-classes

To load a feature layer such as a shapefile or a feature class in a geodatabase into ArcMap entails you to create a feature layer object and then add it into a map object in the document. In Chapter 9, you learned how to obtain an existing layer object from the active (focus) map in the map document object (MxDocument), and you practiced how to access and manipulate the

Name and Visible properties on the ILayer interface of the Layer class. To load a layer into a map takes the other way around – you must first create a new layer object and then add it into the map.

Let's start to analyze the abridged object model diagram in Figure 1 and figure out how to get the job done. Figure 1 shows that the Layer class is an abstract class as indicated by the flat gray box. As an abstract class it cannot instantiate objects by itself. But the Layer class has many child co-classes (see Page 3 of CartoObjectModel.pdf) and you can use any of its child co-classes to instantiate layer objects depending on the layer type. Figure 1 only displays two most frequently used ones: the FeatureLayer co-class and the RasterLayer co-class. If you want to load a feature layer, i.e. vector layer, such as a shapefile or a feature class in a geodatabase, you must use the FeatureLayer co-class to instantiate a feature layer object. If you want to load a raster layer, such as a satellite image, you have to use the RasterLayer co-class to create a raster layer object.

Both the FeatureLayer and the RasterLayer co-classes inherit the Layer abstract class via a series of intermediate classes. This means that these two co-classes inherit the interfaces of all upper-level classes including the Layer abstract class. Therefore, if you instantiate an object from the FeatureLayer co-class, the object can access any property or method on any inherited interface by using Query Interface (QI). And the same is true to objects instantiated from the RasterLayer co-class. The following line instantiates a feature layer object from the FeatureLayer co-class.

```
Dim flayer As IGeoFeatureLayer = New FeatureLayer()
```

where variable *flayer* is declared with the IGeoFeatureLayer interface, and is immediately assigned the address of a new feature layer object. Therefore, variable *flayer* is a proxy of the new feature layer object, and you may sometimes regard it as the object itself. If you want to set a name to this feature layer, you must use the Name property residing on the ILayer interface which is declared in the Layer abstract class. Since the feature layer object inherits the Layer class, you can perform a QI to switch to the ILayer interface in order to assign a name to the layer.

```
Dim layer As ILayer = flayer  ' QI to the ILayer interface
layer.Name = "Parcel"         ' sets the Name property on the ILayer interface
```

Another thing to note in Figure 1 is that the IGeoFeatureLayer interface inherits the IFeatureLayer interface (denoted by IGeoFeatureLayer: IFeatureLayer). This means that the IGeoFeatureLayer interface takes on all properties and method on the IFeatureLayer interface. If you have a feature layer object pointing to the IGeoFeatureLayer interface, you can directly access properties and method inherited from the IFeatureLayer interface without using QI. In this example, feature layer object *flayer* can directly use the inherited FeatureClass property and ShapeType property. For example,

```
' set the FeatureClass property directly to flayer
flayer.FeatureClass = (a feature class object)

' object flayer can directly access the ShapeType property
Msgbox("The shape type of the layer is: " & flayer.ShapeType.ToString())
```

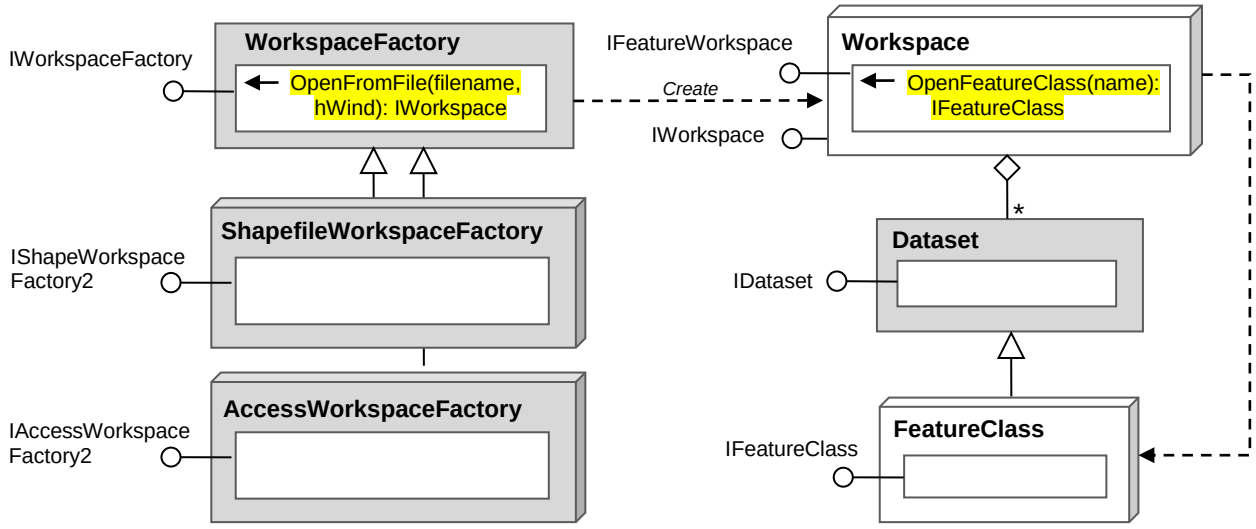
Since we have created a new feature layer object (*flayer*) and set a name to it, can we add it into the map to display the layer now? – Not yet. The feature layer object is still an empty shell without geospatial data. Geospatial data of a feature layer, which consists of spatial and attribute information, is called *feature class*. We must set a feature class object to the FeatureClass property to flesh out the feature layer object. The FeatureClass property is originally declared on the IFeatureLayer interface but is inherited by the IGeoFeatureLayer interface (see Figure 1). If we compare a feature layer object to a CD-ROM disk, a feature class is the data written on it.

A close examination of the FeatureClass property notation in Figure 1.

FeatureClass: IFeatureClass

First, the FeatureClass property is a two-way property. So that you can set a feature class object to it as well as get a feature class object from it. Second, the little hollow box in the barbell ahead of the property name indicates a ByRef property. Normally, a property declared as an object type passes arguments by reference. Finally, the FeatureClass property is declared as an IFeatureClass type. Therefore, when you set the property you must pass a feature class object with IFeatureClass interface to it; and when you get a feature class object from the property, the object you obtain must also point to the IFeatureClass interface.

In order to set the feature class property, we must first create a feature class object. Please analyze the following object model diagram and try to figure out how to create a feature class object (Figure 2).



(Page 1 of GeoDatabaseObjectModel.pdf) (DataSourceGDBObjectModel.pdf)

Figure 2 Abridged object model diagram for creating feature class objects

*In ArcGIS terms, the data of a feature layer is called a **feature class**, and the folder or the geodatabase that holds a feature class is called a **workspace**. In a geodatabase, feature classes may be organized by feature **datasets**. Each feature class is uniquely identified in a geodatabase without having to refer to a feature dataset.*

Figure 2 shows that FeatureClass is a regular class as indicated by the white 3D box. So it cannot be used to instantiate new feature class objects. A workspace object has the ability to create a feature class object since its IFeatureWorkspace interface has a method named OpenFeatureClass() that returns a feature class object with IFeatureClass interface. However, Workspace itself is also a regular class that cannot instantiate new workspace objects. A workspace object must be created by a workspace factory object which has a method named OpenFromFile() on the IWorkspaceFactory interface. The OpenFromFile() method can open a user specified geodatabase or Windows folder and produce a workspace object with IWorkspace interface. However, WorkspaceFactory is an abstract class as indicated by the flat gray box. But it has many child co-classes (see Figure 3). Figure 2 only displays two frequently used co-classes: ShapefileWorkspaceFactory and AccessWorkSpaceFactory. The former can be used to create a workspace factory object to open shapefiles, and the latter can be used to create a workspace factory object that opens Access (personal) geodatabases.

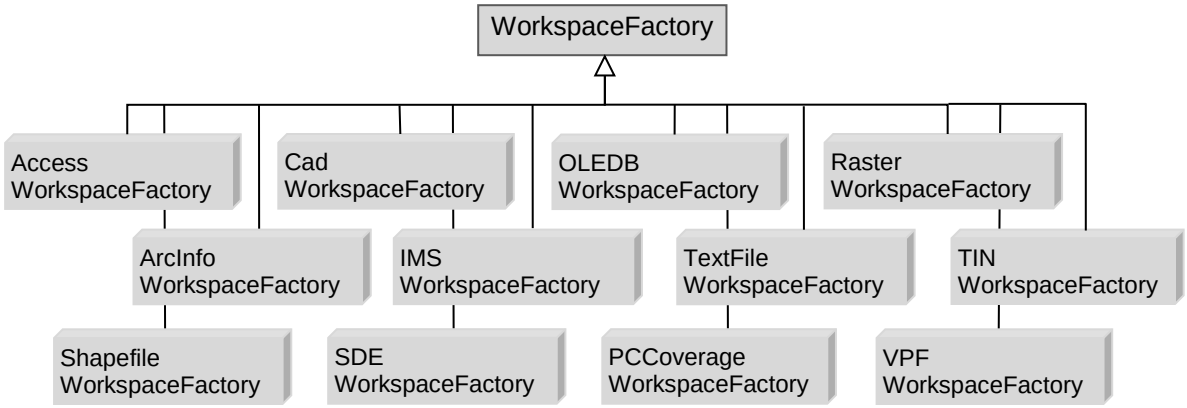


Figure 3 Child co-classes of the WorkspaceFactory abstract class

Based on the object model diagram analysis, we can now draw a roadmap to the creation of a feature layer object.

Step 1: Depending on the type of dataset, e.g. shapefile or personal geodatabase, choose a proper co-class of the WorkspaceFactory class to create a workspace factory object.
Step 2: Call the OpenFromFile() method on the IWorkspaceFactory to create a workspace object. Since the resulting workspace object points the IWorkspace interface, QI is needed to switch to the IFeatureWorkspace interface.
Step 3: Call the OpenFeatureClass() method on the IFeatureWorkspace interface of the workspace object to create a feature class object. The resulting feature class object points to IFeatureClass interface.
Step 4: Use the FeatureLayer co-class to instantiate a new feature layer object that points to the IGeoFeatureLayer interface.
Step 5: Assign the feature class object to the FeatureClass property of the feature layer object.
Step 6: Perform a QI to switch interface from IGeoFeatureLayer to ILayer and then set the Name property of the feature layer object.
Step 7: Add the new feature layer object into a map (e.g. the focus map).

1.2 Creating an add-in button

Suppose the CityGIS project needs to load many feature layers from various datasets residing in different folders and on different drives, manually adding these layers into ArcMap is tedious since the user has to perform many mouse clicks in order to locate one dataset. You are going to create an add-in button on the CityGIS toolbar that loads all feature layers needed by the project with one click.

Please back up your workable CityGIS project before you make changes to it. To maintain multiple versions of the project, you can make a copy of the entire solution folder and rename the copied solution folder with a version number. For example “CityGIS01”, “CityGIS02”, etc.

The following procedure walks you through the process of adding a new add-in buttons (name it “AddLayersButton”) to the CityGIS project.

1) Right click on the CityGIS project name in the Solution Explorer and select Add >> New Item…
2) Select template ArcGIS >> Desktop Add-ins >> Add-in component
3) Enter a file name “AddLayersButton.vb” for the new add-in control
4) In the ArcGIS Add-Ins Wizard, enter “Add Layers” in both caption and tooltip boxes
5) Find image () in the images folder on the CD-ROM and set it to the button
6) Click the Finish button in the Wizard
7) After creating the new button, open the XML document Config.esriaddinx
8) Insert a new button element in the Items element by following the example of existing buttons. Make sure the refID attribute value is copied from the new button’s id in the Commands element of the document.

```
    <ComboBox id="NAU_CityGIS_MapScalesComboBox" class="MapScalesComb
    <Button id="NAU_CityGIS_AddLayersButton" class="AddLayersButton"
  </Commands>
  <Toolbars>
    <Toolbar id="NAU_CityGIS_City_GIS_Toolbar" caption="City GIS Tool
      <Items>
        <Button refID="NAU_CityGIS_AddLayersButton" />   <-- Insert the new button here
        <Button refID="NAU_CityGIS_LayersOnOffButton" />
        <Button refID="NAU_CityGIS_NewBusStopFormButton" />
```

9) Run the program to check if the new button is successfully added into the toolbar in ArcMap (Figure 4).

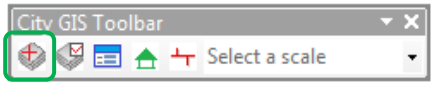


Figure 4 A new button is added into the toolbar

1.3 Implementing the load layer button

To implement the load layer button, let us start with adding one feature class from a personal geodatabase. Please find a personal geodatabase on the CD-ROM at Data\Flagstaff.mdb. Suppose your CD or DVD drive is installed as E drive, the full path to the personal geodatabase is E:\Data\Flagstaff.mdb. You are going to write code to load a feature class named “Parcel” in the Land feature dataset in this geodatabase.

Before writing code, please load the following ArcObjects assemblies into your CityGIS project as references:

- ESRI.ArcGIS.Geodatabase assembly (contains WorkspaceFactory, Workspace classes)
- ESRI.ArcGIS.DataSourcesGDB assembly (contains AccessWorkspaceFactory class)
- ESRI.ArcGIS.Display (contains color and map symbol classes, also required by Carto)

The first assembly includes the WorkspaceFactory and Workspace classes (see model diagrams in GeoDatabaseObjectModel.pdf), and the second assembly includes the AccessWorkspaceFactory class (see model diagrams in DataSourceGDBObjectModel.pdf). The Display assembly contains color and map symbol classes. Besides, the Carto assembly has a dependency on the Display assembly in creating feature layer objects. After loading these two assemblies, you can import the following namespaces in the header section of the AddLayersButton class module.

```
Imports ESRI.ArcGIS.ArcMapUI        ' needed by MxDocument class etc.
Imports ESRI.ArcGIS.Carto           ' needed by Map and Layer classes etc.
Imports ESRI.ArcGIS.Geodatabase     ' needed by WorkspaceFactory and Workspace classes
Imports ESRI.ArcGIS.DataSourcesGDB  ' needed by AccessWorkspaceFactory class
```

Once necessary assemblies are added into the reference and namespaces are imported in the code module, you can implement the OnClick event of the AddLayersButton class by using the code below. Please read the code and comments (green text) carefully. You may need to repeatedly check back on the model diagrams and discussions in previous sections when you read the code.

```
Protected Overrides Sub OnClick()
    ' get the map document object from ArcMap (analogous to an airport)
    Dim mxDoc As IMxDocument = My.ArcMap.Document

    ' get the focus map (with IMap interface) and use a variable (map) to hold it
    Dim map As IMap = mxDoc.FocusMap

    ' Step 1:
    ' create a workspace factory object from co-class AccessWorkspaceFactory
    ' and assigns it to variable wksFactory
    Dim wksFactory As IWorkspaceFactory = New AccessWorkspaceFactory

    ' Step 2:
    ' call the OpenFromFile() method of the workspace factory object (wksFactory)
    ' to create a workspace object and assign it to new variable wks. QI implied
    Dim wks As IFeatureWorkspace = _
            wksFactory.OpenFromFile("E:\Data\Flagstaff.mdb", 0)

    ' Step 3:
    ' call the OpenFeatureClass() method of the workspace object (wks) to
    ' create a feature class object and assign it to new variable fclass
    Dim fclass As IFeatureClass = wks.OpenFeatureClass("Parcel")

    ' Step 4:
    ' create a new feature layer object and assign it to new variable flayer.
```

```
        ' this new feature layer object is an empty shell without data at this point
        Dim flayer As IGeoFeatureLayer = New FeatureLayer

        ' Step 5:
        ' set the feature class object (fclass) to the FeatureClass property
        ' of the feature layer object (flayer)
        flayer.FeatureClass = fclass

        ' Step 6:
        ' QI to the ILayer interface (note flayer points to IGeoFeatureLayer interface)
        Dim layer As ILayer = flayer
        ' set a name to the Name property of the new layer
        layer.Name = "Flagstaff Parcels"

        ' Step 7:
        ' call the AddLayer() method of the map object to add the new layer
        map.AddLayer(layer)

        ' Finally, refreshes the map and update the TOC
        mxDoc.ActivatedView.Refresh()
        mxDoc.UpdateContents()
    End Sub
```

While most of the steps are straightforward, steps 2 and 3 need some additional explanation. In step 2, the OpenFromFile() method on the IWorkspaceFactory interface opens a workspace specified by the client. A workspace can be a personal geodatabase, a file geodatabase, or a folder containing shapefiles. The OpenFromFile() method has two parameters: a string for the workspace to open and an integer for the handle of the parent window or application's window. Because we are loading a feature class from a personal geodatabase, the full path of the geodatabase "E:\Data\Flagstaff.mdb" is passed to the method as the first argument in step 2. For the second parameter of the method, you can always pass 0 to it so that if anything goes wrong when the method tries to open the workspace, a dialog will be displayed in the parent window. Please also note that the OpenFromFile() method returns a workspace object with IWorkspace interface. But in step 2, the result of the method is assigned to a variable declared to point to the IFeatureWorkspace interface. A Query Interface is implicitly performed during the value assignment in this step.

In step 3, the OpenFeatureClass() method opens a feature class from a workspace by a name provided by the client. The method requires one argument for the feature class name, so feature class "Parcel" is passed to it. Since each feature class is uniquely identified in a geodatabase, even if a feature class is placed in a feature dataset, the feature dataset does not have to be reference.

1.4 Enhancement

The load layer button should work if you entered the code correctly and provided correct workspace and feature class names, although it only loads the parcel feature class so far. If you want it to load more layers by this button, you could copy the code, paste it into the same OnClick event procedure, and edit the workspace and feature class names. But such a solution is awkward and inefficient. A smart way is to create a function that can create a feature layer object based on a workspace name and a feature class name passed to it as arguments. Then you can call this function as many times to load multiple layers in the button click event

procedure. At each call, you pass a workspace name and a feature class name to the function, and you get a layer object from it. Then you can add the layer into the map to display. The following is a function named CreateFeatureLayer() that satisfies this need. This function is implemented by the code in step 1 through 6 in the previous button click event procedure. But the hard-coded workspace (database) name and feature class name are replaced by parameters of the function.

```
Private Function CreateFeatureLayer(DatabaseName As String,
                                    FeatureClassName As String) As IGeoFeatureLayer
    ' check if the client-provided geodatabase (a file) exists
    If Not IO.File.Exists(DatabaseName) Then Return Nothing

    ' create an access workspace factory object
    Dim wksFactory As IWorkspaceFactory = New AccessWorkspaceFactory
    ' call the OpenFromFile() to create a workspace object. QI implied.
    Dim wks As IFeatureWorkspace = wksFactory.OpenFromFile(DatabaseName, 0)

    ' declare a new variable for IFeatureClass interface
    Dim fclass As IFeatureClass = Nothing
    Try
        ' call the OpenFeatureClass() method to create a feature class object
        fclass = wks.OpenFeatureClass(FeatureClassName)
    Catch ex As Exception
        Return Nothing  ' if the OpenFeatureClass() method fails, return nothing
    End Try

    ' create a new feature class object with IGeoFeatureLayer interface
    Dim flayer As IGeoFeatureLayer = New FeatureLayer()
    flayer.FeatureClass = fclass    ' sets its FeatureClass property

    Dim layer As ILayer = flayer    ' QI to ILayer interface
    layer.Name = FeatureClassName   ' sets a name to the layer

    Return flayer   ' returns a feature layer object to the client
End Function
```

In this function that two features are added to enhance reliability and avoid crashing. First, it uses the VB built-in function IO.File.Exists() to check existence of the client provided geodatabase (.mdb file). For file geodatabase and shapefile workspace, you need to check existence of a folder workspace. Then you can use function IO.Directory.Exists(). Second, the function uses a Try structure to prevent the OpenFeatureClass() method from crashing which happens frequently when the user provided feature class name does not exist in the workspace.

After creating function CreateFeatureLayer(), the button click event procedure can be substantially simplified and the code becomes much neat and elegant.

```
Protected Overrides Sub OnClick()
    Dim mxDoc As IMxDocument = My.ArcMap.Document  ' gets the map document object
    Dim map As IMap = mxDoc.FocusMap  ' gets the focus map object

    Dim flayer As IGeoFeatureLayer    ' declares a feature layer variable
    ' call the CreateFeatureLayer() function to create a feature layer object
    flayer = CreateFeatureLayer("E:\Data\Flagstaff.mdb", "Parcel")
    map.AddLayer(flayer)              ' adds the layer into map
```

```
        mxDoc.ActivatedView.Refresh()      ' refreshes the map
        mxDoc.UpdateContents()             ' updates TOC
    End Sub
```

After modifying the button click event procedure, run the program to check if the button can load the parcel feature class into ArcMap.

The CreateFeatureLayer() function you just created is very useful. It can be called to load any feature class existing in any personal geodatabase. To add another layer with this function, you only need to add two lines of code after adding the first layer and before refreshing the map in the button click event procedure. For example, to load the Building layer:

```
        flayer = CreateFeatureLayer("E:\Data\Flagstaff.mdb", "Building")
        map.AddLayer(flayer)      ' adds the layer into map
```

As you can see in the above code, you can even reuse the existing variable *flayer* to hold the new layer object returned from the CreateFeatureLayer() function.

2. Loading Raster Layers

Functions and sub procedures are important mechanisms to break a complex job down into smaller tasks. As demonstrated by the loading feature layer example, custom functions are extremely helpful since they can effectively reduce code redundancy and they make the logic of workflows clear. As a reusable code unit, the CreateFeatureLayer() function can be called whenever you need to create a feature layer object. Moreover, since the CreateFeatureLayer() function takes charge of creating new feature layer objects, the button's Click event procedure can focus on adding new layers into the map.

To load a raster layer, we will start with creating a function (let's name it *CreateRasterLayer*()) that returns a raster layer object. Then we will call this function in the load layer button's Click event procedure to generate raster layer objects, similar to loading feature layers. To create such as function, we need to analyze the model diagram first.

2.1 Model diagram analysis and solution development

Figure 5 is quite similar to Figure 1. The RasterLayer class is a co-class that can instantiate new raster layer objects. Like the FeatureLayer class, a new raster layer object instantiated from the RasterLayer class is also an empty shell. Yu need to call the CreateFromDataset() method on the IRasterLayer interface to set a raster dataset into the raster layer. A raster dataset for a raster layer is equivalent to the feature class for a vector layer. It is the actual raster data. Since the CreateFromDataset() method requires a raster dataset object as an input argument, you need to create a raster dataset object before calling the method. Similar to a FeatureClass class, however, the RasterDataset class is also a regular class that cannot instantiate new objects by itself. You have to use a raster workspace object to create a raster dataset object. But the RasterWorkspace class is a regular class that does not instantiate new objects by itself. Finally, you need a workspace factory object to create a raster workspace object.

Fortunately, the WorkspaceFactory abstract class has many co-classes among which the RasterWorkspaceFactory can be used to instantiate a workspace factory object that satisfies our need. The OpenFromFile() method on the IWorkspaceFactory interface takes the full path of the directory (folder) that contains the raster file as input and returns a workspace object. The IRasterWorkspace interface of the workspace object has a method named OpenRasterDataset() which takes in a raster file name as input and returns a raster dataset object. Eventually, we can take a very similar procedure to create a raster layer object as we did in creating a feature layer object.

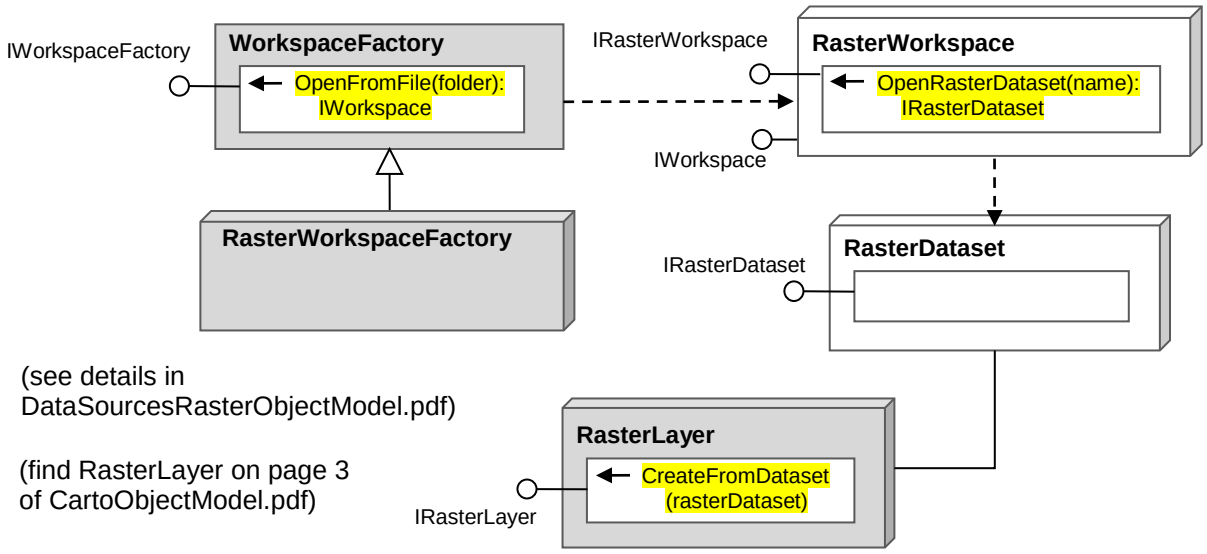


Figure 5 Object model diagram for creating raster datasets and raster layers

Step 1: Use the RasterWorkspaceFactory co-class to instantiate a workspace factory object.
Step 2: Call the OpenFromFile() method of the workspace factory object to create a raster workspace object. Since the resulting workspace object points to the IWorkspace interface, a QI needs to be performed to get the IRasterWorkspace interface in order to access the OpenRasterDataset() method.
Step 3: Call the OpenRasterDataset() method on the IRasterWorkspace interface of the raster workspace object to create a raster dataset object.
Step 4: Create a new rater layer object from the RasterLayer co-class and assign a name to its Name property. Note that the new raster layer object is an empty shell.
Step 5: Call the CreateFromDataset() method on the IRasterLayer interface of the raster layer object to load the raster dataset object created in Step 3 into the raster layer.

2.2 Creating a function to load raster layers

To implement the CreateRasterLayer() function you must load the DatasourcesRaster assembly into your CityGIS project as a reference because it contains the RasterWorkspaceFactory class. Then you can import its namespace (ESRI.ArcGIS.DataSourceRaster) into the AddLayersButton class module.

The code for the CreateRasterLayer() function is provided below. This function declares two String type parameters. The first parameter *FolderName* is used to receive the full path of a

folder that contains the raster dataset. The incoming value of this parameter is used by the OpenFromFile() method to create a raster workspace object in step 2. The second parameter *RasterFileName* is used to receive the name of the raster file. The incoming value of this parameter is used in step 3 by the OpenRasterDataset() method to create a raster dataset object. When you call this function to create a raster layer object, you must provide two string values to it: the full path of the folder that contains the raster file, and the name of the raster file.

```
Private Function CreateRasterLayer(ByVal FolderName As String,
                                   ByVal RasterFileName As String) As IRasterLayer
    ' check folder (workspace) existence
    If Not IO.Directory.Exists(FolderName) Then Return Nothing
    ' check raster file existence
    If Not IO.File.Exists(FolderName & "\" & RasterFileName) Then Return Nothing

    ' Step 1:
    ' call the RasterWorkspaceFactory co-class to create a workspace factory object
    Dim wksFactory As IWorkspaceFactory = New RasterWorkspaceFactory()

    ' Step 2:
    ' call the OpenFromFile method of workspace factory to create a workspace object.
    ' since the method returns a workspace object with IWorkspace interface, and
    ' the resulting object is assigned to a new variable rstwks pointing to the
    ' IRasterWorkspace interface, a QI is implied in the following line
    Dim rstwks As IRasterWorkspace = wksFactory.OpenFromFile(FolderName, 0)

    ' Step 3:
    ' call the OpenRasterDataset method of the raster workspace object to
    ' create a raster dataset object and assign it to new variable rstDataset
    Dim rstDataset As IRasterDataset = rstwks.OpenRasterDataset(RasterFileName)

    ' Step 4:
    ' instantiate a new raster layer object from the RasterLayer co-class
    Dim rlayer As IRasterLayer = New RasterLayer
    rlayer.Name = RasterFileName    ' sets the Name property of the raster layer
    ' Note: IRasterLayer interface inherits the ILayer interface (see Figure 1)

    ' Step 5:
    ' call the CreateFromDataset method of the raster layer object
    ' to load the raster dataset object into the raster layer
    rlayer.CreateFromDataset(rstDataset)

    Return rlayer   ' return the raster layer to the calling line.
End Function
```

The function first checks existence of the workspace (folder) and raster file received by the parameters. Existence of the folder and file does not mean the file is a valid raster. I would like you to write a try structure to test code in step 3 to enhance reliability. To call this function to load a raster layer, please add the following three lines into the button's Click event procedure, below code line "`Dim map ...`" and above code line "`Dim flayer ...`"

```
    Dim rlayer As IRasterLayer  ' declares a variable for raster layer
    ' call the CreateRasterLayer() function to create a raster layer
    rlayer = CreateRasterLayer("E:\Data\Satellite", "Ponderosa.jp2")
    map.AddLayer(rlayer)
```

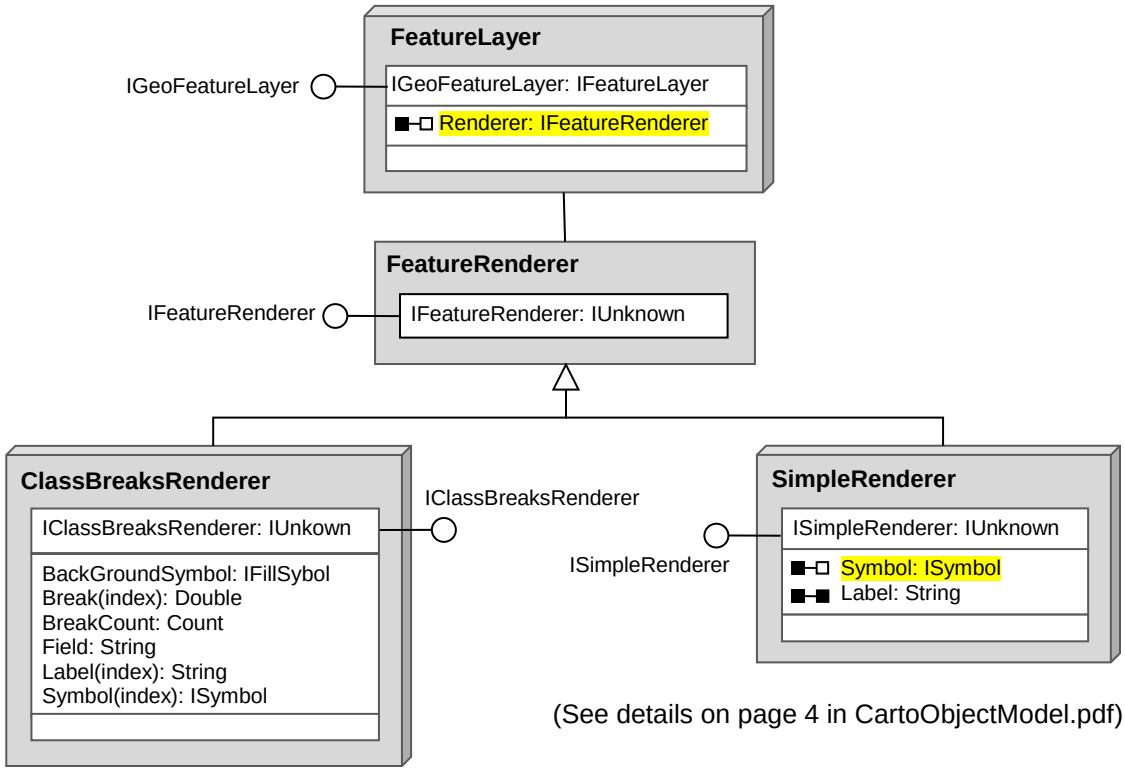
Make sure the workspace and image file exist on your computer and start the program to test the add layers button and see if it loads the image layer.

3. Rendering Feature Layer

In section 1 of this chapter you loaded two feature layers without assigning colors to them so ArcMap randomly assigned colors to the layers. You can assign a color to a feature layer as soon as it is created just like to set the Name property. Unlike setting the Name property which is a simple string, however, rendering to a feature layer involves creating a number of objects.

In this section you will learn how to create simple renderers and map symbols to render feature layers. Before writing code, please study the object model diagrams.

3.1 Object model diagram analysis and solution development



(See details on page 4 in CartoObjectModel.pdf)

Figure 6 Object model diagram for rendering feature layers

First, the Renderer property on the IGeoFeatureLayer interface of the FeatureLayer class is declared as IFeatureRenderer interface which means you can set a feature renderer object to it (note it is a two-way property). In order to set the Renderer property, you must first create a feature renderer object.

Second, to instantiate a feature renderer object, you need to use an appropriate co-classes of the FeatureRenderer abstract class. The FeatueRenderer abstract class has many co-classes but Figure 6 only shows two frequently used ones. In this example, you will use the

SimpleRenderer co-class to instantiate a simple renderer object. A simple renderer object renders a feature layer using one symbol. For example, if the layer is a point or polygon layer, it draws all point or polygon features with one fill color and one border style. If the layer is a line layer, it draws all line features with one color and one width.

Third, a simple renderer object requires you to set the Symbol property which takes a symbol object (with ISymbol interface). Therefore, you must create a symbol object before setting this property.

Fourth, the Symbol class is an abstract class. Depending on the shape type (point, line, or polygon) of the feature layer, you need to choose a proper co-class to instantiate a symbol object (Figure 7). To render the polygon layers Parcel and Building, you will use the SimpleFillSymbol co-class. The ISimpleFillSymbol interface inherits the IFillSymbol interface, which means that the ISimpleFillSymbol interface takes on the Color property and Outline property. In order to set the Outline property, you must instantiate a line symbol object from the SimpleLineSymbol co-class. Since both FillSybol and LineSymbol classes have a Color property, you must create two color objects.

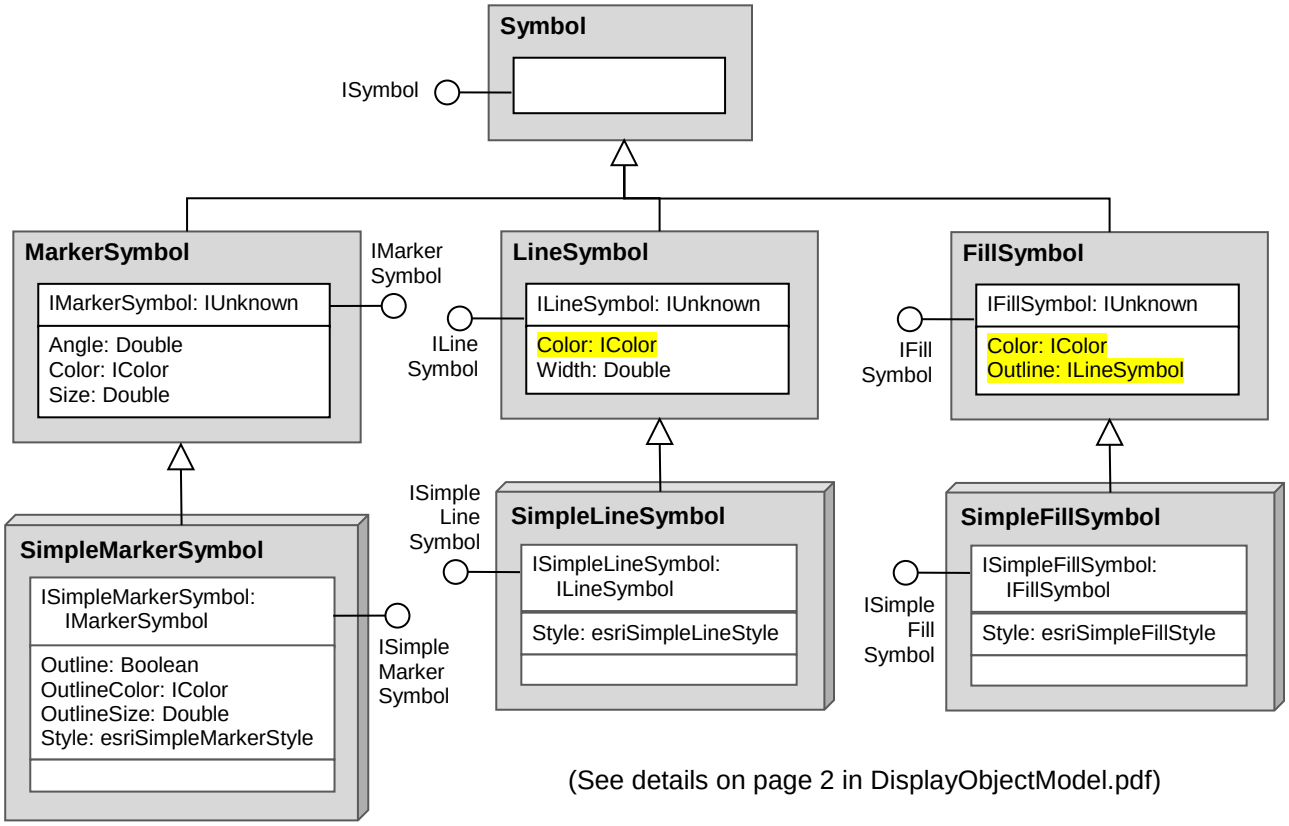


Figure 7 The Symbol abstract class and its child classes

Finally, to create a color object, you can normally use the RgbColor co-class (Figure 8). Once you have instantiated an RgbColor object, you can set its Red, Green, and Blue properties. Each of these properties takes an integer number between 0 and 255 to indicate the amount of red, green or blue light. You can easily find explanations and tools for RGB colors on the web. For example, you can use the color wheel at http://www.colorspire.com/rgb-color-wheel/ to find the red, green, and blue values of any color.

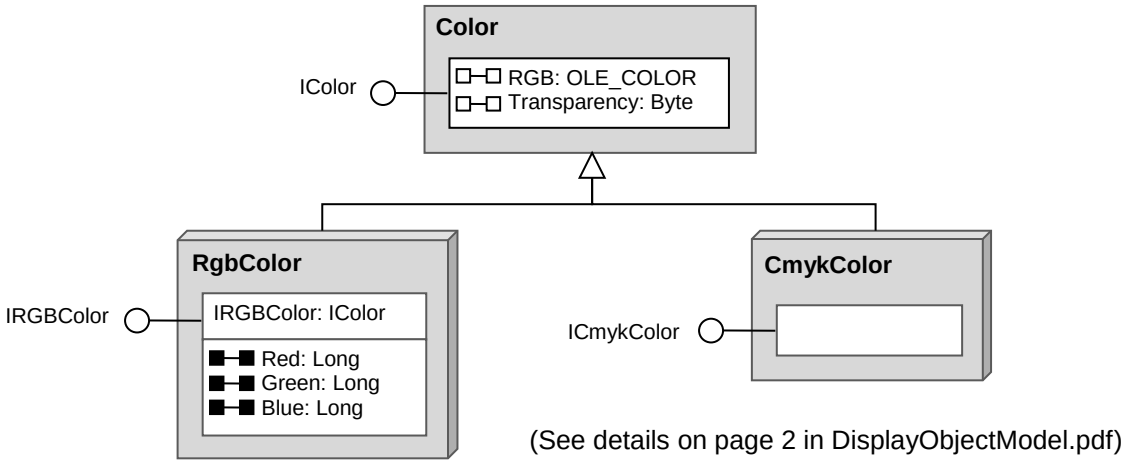


Figure 8 The Color abstract class and two child co-classes

In summary, in order to create a renderer object to render a polygon layer, you need to create two color objects – one for the fill color and the other for border color, a simple line symbol object which will consume the border color, and a simple fill symbol object which will consume the fill color and the line symbol objects. Once a simple fill symbol object is ready, you instantiate a simple renderer object from the SimpleRenderer co-class and set the the simple fill symbol object to its Symbol property.

3.2 Creating a simple fill renderer

This job will be accomplished by creating four custom functions: CreateColor(), a CreateLineSymbol(), CreateFillSymbol(), and CreateSimpleRenderer(). The CreateColor() function will declare three integer parameters, one for a primary color, and return an RGB color object. The CreateLineSymbol() function will take a color object and a width as input and return a simple line symbol object. The CreateFillSymbol() function will take two arguments, a color object and a line object, as input and return a simple fill symbol object. The CreateSimpleRenderer() function will take a fill symbol object as input and return a simple feature renderer object.

In order to create these functions, you must add the Display assembly into your project as a reference (right click on Reference in the Solution Explorer…) because the Symbol and Color classes are in this assembly. You also need to import the ESRI.ArcGIS.Display namespace in the header section of the AddLayersButton class module. The functions CreateColor(), CreateLineSymbol() and CreateFillSymbol() are implemented as the following.

```
Private Function CreateColor(red As Integer,
                             green As Integer,
                             blue As Integer) As IRgbColor
    ' create a new RgbColor object from the RgbColor co-class
    Dim color As IRgbColor = New RgbColor
    color.Red = red     ' sets red property
    color.Green = green ' sets green property
    color.Blue = blue   ' sets blue property
    Return color        ' returns the result
End Function
```

```
Private Function CreateLineSymbol(lineColor As IColor,
                                  lineWidth As Double) As ILineSymbol
    ' instantiate a line symbol object from the SimpleLineSymbol co-class
    Dim lineSymbol As ILineSymbol = New SimpleLineSymbol()
    lineSymbol.Color = lineColor      ' sets color property
    lineSymbol.Width = lineWidth      ' sets width property
    Return lineSymbol                 ' returns the result
End Function

Function CreateFillSymbol(fillColor As IColor,
                          lineSymbol As ILineSymbol) As ISimpleFillSymbol
    Dim sfSymbol As ISimpleFillSymbol = New SimpleFillSymbol()
    sfSymbol.Color = fillColor
    sfSymbol.Outline = lineSymbol
    sfSymbol.Style = esriSimpleFillStyle.esriSFSSolid
    Return sfSymbol
End Function
```

When you read the code of these functions against the class diagram of each component, you will find they are very straightforward. In fact, writing code based on a single class is not be as intimidating as the complete object model diagrams may suggest. Once a fill symbol is available, implementing the CreateSimpleRenderer() function becomes even simple.

```
Private Function CreateSimpleRenderer(fillSymbol As IFillSymbol) _
                 As ISimpleRenderer
    ' create a new simple renderer object from SimpleRender co-class
    Dim srndr As ISimpleRenderer = New SimpleRenderer()

    ' set the fill symbol to the Symbol property
    srndr.Symbol = fillSymbol

    ' return the simple renderer object
    Return srndr
End Function
```

Making a simple fill render object is similar to building a Lego transformer in which you first snap a few blocks together to make a part such as an arm or a leg. Once you have created all necessary parts, you can put them together to make a hero.

Once the CreateSimpleRenderer() function is created, you can call it in the add layers button's Click event procedure to create renderers for the parcel and building feature layers. Please replace the code that creates and displays the two feature layers by the following code:

```
' create a feature layer by calling the CreateFeatureLayer function
Dim flayer As IGeoFeatureLayer
flayer = CreateFeatureLayer("E:\Data\Flagstaff.mdb", "Parcel")
If flayer IsNot Nothing Then
    Dim fColor As IColor = CreateColor(215, 255, 225)        ' fill color: pale green
```

```
        Dim bColor As IColor = CreateColor(127, 127, 127)        ' border color: gray
        Dim lineSym As ILineSymbol = CreateLineSymbol(bColor, 1) ' creates a line symbol
        Dim fillSym As IFillSymbol = CreateFillSymbol(fColor, lineSym)  ' fill symbol
        flayer.Renderer = CreateSimpleRenderer(fillSym)
        map.AddLayer(flayer)     ' adds the layer into map
    End If

    flayer = CreateFeatureLayer("E:\Data\Flagstaff.mdb", "Building")
    If flayer IsNot Nothing Then
        Dim fColor As IColor = CreateColor(255, 127, 127)         ' fill color: pink
        Dim bColor As IColor = CreateColor(127, 127, 127)         ' border color: gray
        Dim lineSym As ILineSymbol = CreateLineSymbol(bColor, 1)  ' creates a line symbol
        Dim fillSym As IFillSymbol = CreateFillSymbol(fColor, lineSym)  ' fill symbol
        flayer.Renderer = CreateSimpleRenderer(fillSym)
        map.AddLayer(flayer)     ' adds the layer into map
    End If
```

After calling the CreateFeatureLayer() function, the code uses an If block to check if a valid feature layer object is returned from the function. If a feature layer object is successfully created, the program creates two color objects (fColor for fill color and bColor for border color) and a line symbol object (lineSym for polygon borders), and then calls the CreateSimpleRenderer() function to create a simple renderer object. Finally, the renderer object is assigned to the Renderer property of the new feature layer object, and the feature layer is added into the map for display. Run the program and test the add layers button.

Chapter Summary and Highlights

To display a feature layer in ArcMap by code, you need to create a feature layer object. The geospatial data of a feature layer is called feature class. You must create a feature class object and set it to the feature class property of the feature layer object in order to display the layer.

Creating a feature class object needs several steps and involves a number of ArcObjects components and interfaces. First, depending on the type of data, you must choose an appropriate workspace factory co-class to instantiate a workspace factory object. Then you need to call the OpenFromFile() method on the IWorkspaceFactory interface to open a workspace and create a workspace object. Finally, you can call the OpenFeatureClass() method on the IFeatureWorkspace interface of the workspace object to create a feature class object.

The procedure to display a raster layer in ArcMap is very similar to displaying a feature layer. You need to create a raster layer object first and call the CreateFromDataset() method on the IRasterLayer interface to load a raster dataset object into the layer. A raster dataset object is the actual data of a raster layer. To create a raster dataset object you need to start with the instantiation of a workspace factory object using the raster workspace factory co-class. Then you can call the OpenFromFile() method on the IWorkspaceFactory interface to open a workspace (e.g. a folder containing the raster) and produce a workspace object. Finally, you can call the OpenRasterDataset() method on the IRasterWorkspace interface to produce a raster dataset object.

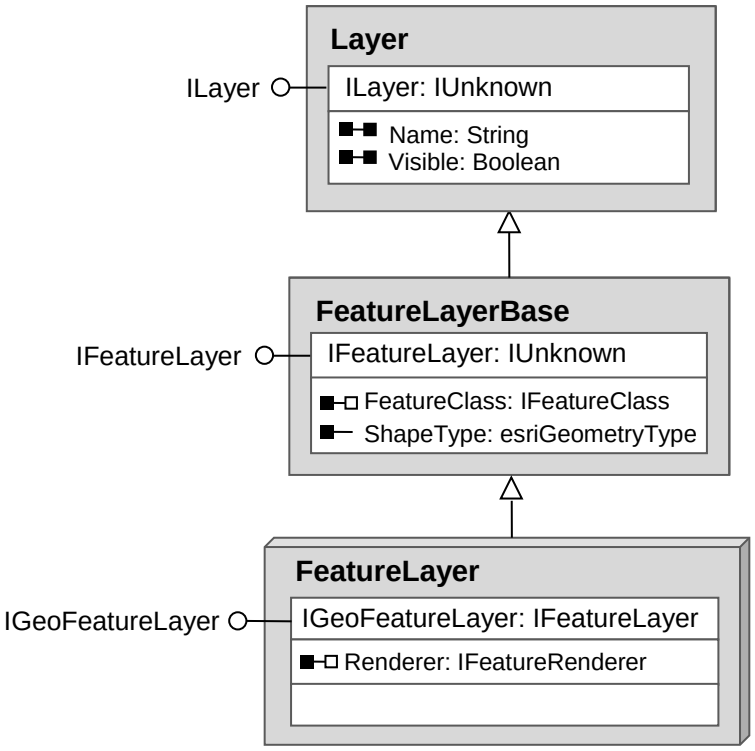
To render a feature layer entails you to create a renderer object. To create a renderer object requires symbol objects such as marker symbols, line symbols, and fill symbols depending on

the shape type of the feature layer. All these symbols require color objects. To effectively render feature layers, you first create some functions that make basic symbol objects such as colors, line symbols, and fill symbols. With these foundational functions, you can assemble more complex symbols and renderers like playing Legos. In fact, programming with ArcObjects is quite like playing Legos.

Exercises

1. Answer the following questions.
    1) What class do you use to instantiate a new feature layer object?
    2) What class do you use to instantiate a new raster layer object?
    3) What classes does the FeatureLayer class inherit?
    4) Which interface do you use to set a name to a layer?
    5) Which interface do you use to set a feature class object to a feature layer?
    6) How do you create a feature class object?
    7) Why can the IGeoFeatureLayer interface directly access the FeatureClass property that resides on the IFeatureLayer interface without using QI?
    8) What is interface inheritance? How is interface inheritance denoted in object model diagrams?
    9) What does a hollow box in the barbell ahead a property indicate?
    10) What is a ByRef property?
    11) What method on what interface can be used to create a feature class object?
    12) What method on what interface can be used to create a workspace object?
    13) What co-class can be used to instantiate a workspace factory object that opens a personal geodatabase?
    14) What co-class can be used to instantiate a workspace factory object that opens a shapefile?
    15) Which ArcObjects assembly includes the WorkspaceFactory and Workspace classes?
    16) Which ArcObjects assembly includes the AccessWorkspaceFactory class?
    17) What is the purpose to add import statements in the header of the code editor?
    18) What does function IO.File.Exists() do?
    19) What does the CreateFromDataset() method of the RasterLayer class do?
    20) What are the arguments for the OpenFromFile() method on the IWorkspaceFactory?
    21) How do you create a color object?
    22) How do you create a line symbol object?
    23) How do you create a simple fill symbol object?
    24) How do you create a simple renderer object?
    25) How do you render a feature layer with a simple renderer?

2. Analyze the following object model diagram and answer questions.



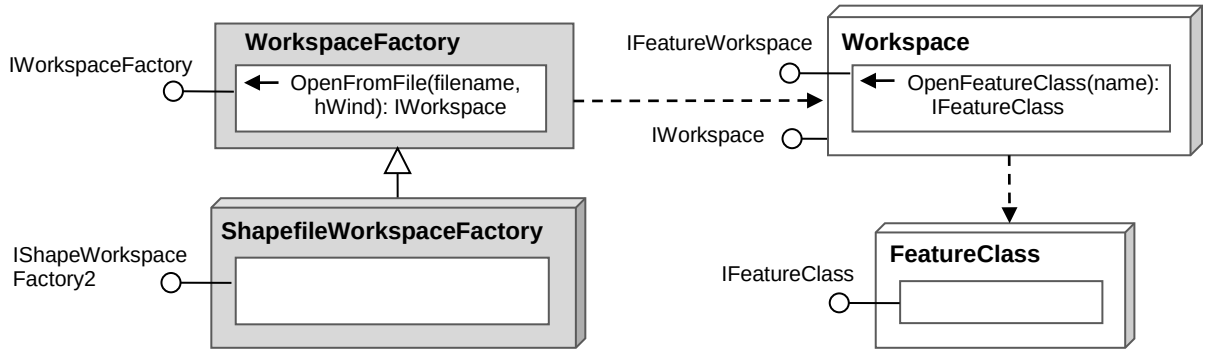
1) Write code to instantiate a feature layer object and store (the address of) it in a new variable named *flyr* that points to the IGeoFeatureLayer interface.

2) Given the feature layer object (represented by variable *flyr*) you created in question 1, what properties can you access via this variable according to the ArcObjects model diagram above?

3) Suppose a feature class object is represented by variable *fclass* that points to IFeatureClass interface, write code to set the FeatureClass property of the feature layer object (flyr) you created in question 1 and use a message box to report the shape type of the layer.

4) Given a feature layer object (represented by variable *flyr*) you created in question 1, write code to set a name “Layer 1” to the layer object and set the visibility property to False.

3. Analyze the following ArcObjects model diagram and answer questions.



   1) Can you instantiate objects from all these classes? Why?

   2) Write code to instantiate a shapefile workspace factory object and store (the address of) it in a new variable *shpwksfact* that points to the IShapeWorkspaceFactory2 interface.

   3) Given a shapefile workspace "D:\Data\Shapefiles", use the workspace factory object you created in the previous step to create a workspace object and store the resulting object in a new variable *wks* that points to IWorkspace interface.

   4) Given a shapefile named "Roads.shp", use the workspace object you created in the previous step to create a feature class object and store the resulting object in a new variable *fclass* that points to the IFeatureClass interface.

4. The CityGIS project can now load any feature layer in a personal geodatabase. Back up your CityGIS add-in project, then add code in the add layers button Click event procedure to load the Streets feature class in the Flagstaff.mdb personal geodatabase. To renderer the street layer, you need write code to overload the CreateSimpleRenderer() function in the AddLayersButton class. The overloading function will take a line symbol object (with ILineSymbol interface) as input and return a simple renderer object (ISimpleRenderer interface).

```
Private Function CreateSimpleRenderer(lineSymbol As ILineSymbol) As ISimpleRenderer

End Function
```

   Create a simple renderer object to render the street layer with gray color (RGB values 127, 127, 127) and width 1.0.

5. Back up your CityGIS add-in project and create a new function CreateShapeLayer() function and use it to load the waterline shapefile (Waterline.shp) and fire hydrant shapefile (Fire_Hydrants.shp) in folder Data\Shapefiles on the CD-ROM. Call function CreateSimpleRenderer() you create in Problem 4 to render the waterline layer with a blue color (RGB: 0, 0, 255) and with 1.25.

   To implement the function, you must have a thorough understanding of section 1 of Chapter 10, especially the Figures 1 and 2. This function will be very similar to the CreateFeatureLayer() function except that it opens a shapefile. More suggestions and tips:

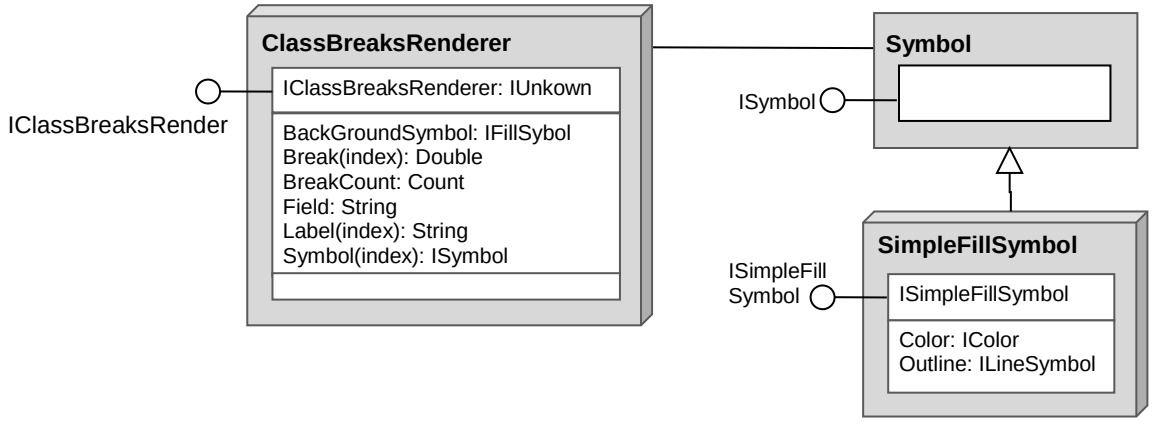
- The function should take two arguments: one for the folder name and the other for the shapefile name. It should return a feature layer object with IGeoFeatureLayer interface. So the declaration of the function may look like:

```
Private Function CreateShapeLayer(Folder As String,
                                  Shapefile As String) As IGeoFeatureLayer

End Function
```

- You need to use the ShapeWorkspaceFactory co-class to create a workspace factory object. Since the ShapeWorkspaceFactory class is included in the DataSourceFile assembly, you must load this assembly into the project as a reference and import the ESRI.ArcGIS.DataSourceFile namespace into the class module.
- The workspace of a shapefile is the folder that contains that shapefile. You should pass the full path of the folder to the OpenFromFile() method on the IWorkspaceFactory interface as an argument.
- You can use a VB built-in function IO.Directory.Exists() to check folder existence, and use function IO.File.Exists() to check shapefile existence (full path required).
- The OpenFeatureClass() methods on the IFeatureWorkspace interface can take the shapefile name (not including path, with/without file extension ".shp") as input.

6. Create a feature renderer object to render the population of the US States shape layer. The renderer will use the "Pop1990" field to break state populations into 5 classes with the following breaks: 2000000, 4000000, 8000000, 16000000, and 50000000. Each class will be rendered with a fill color whose red value is defined by formula (127 + i * 40) and both green and blue values are defined by formula (i * 32) where i is the class number that runs from 0 to 4. Analyze the object model diagram below and follow steps to accomplish the task. The *Break* property, *Symbol* property and *Label* property are arrays with upper bound 4, i.e. array sizes are same as the number of classes.



1) Instantiate a class breaks renderer object and then set the BreakCount property to 5, the Field property to "Pop1990", and the Break property which is an array.

```
' Instantiate a class breaks renderer
Dim clsbreak As IClassBreaksRenderer = <your code>
```

```
' Set some known properties
clsbreak.Field = <your code>
clsbreak.BreakCount = <your code>

' Set the Break property
clsbreak.Break(0) = 2000000
clsbreak.Break(1) = <your code>
clsbreak.Break(2) = <your code>
clsbreak.Break(3) = <your code>
clsbreak.Break(4) = <your code>
```

2) In order to set the Symbol property which is an array, you need to create five fill symbol objects. Use the three functions CreateColor(), CreateLineSymbol(), and CreateFillSymbol() to create an array of five fill symbols.

```
' Create a border color (75% gray)
Dim lineColor As IColor = CreateColor(192, 192, 192)

' Create a line symbol for polygon borders
Dim lineSymbol As ILineSymbol = CreateLineSymbol(lineColor, 1.5)

' Declare an array variable for five color objects
Dim fillColor(4) As IColor
Dim R, G, B As Integer

' Declare an array variable for five fill symbol objects
Dim fillSymbl(4) As IFillSymbol

' Use a loop to create five fill colors and five fill symbols
For i As Integer = 0 To 4
      R = <your code>
      G = <your code>
      B = <your code>
      fillColor = <call the CreateColor() function>
      fillSymbol(i) = <call the CreateFillSymbol() function>
Next

' Finally, set the Symbol property of the class breaks renderer object
For i As Integer = 0 to 4
      clsbreak.Symbol(i) = <your code>
Next
```